Advanced machine learning

Graph neural networks

Mikkel N. Schmidt

Technical University of Denmark,
DTU Compute, Department of Applied Mathematics and Computer Science.

Overview

1. Module overview
2. Results from last week's competition
3. Message passing on graphs
   - Neighborhood aggregation
   - Update methods
   - Graph pooling
   - Machine learning tasks on graphs
4. Geometric embedding
5. Pytorch implementation
6. Exercises

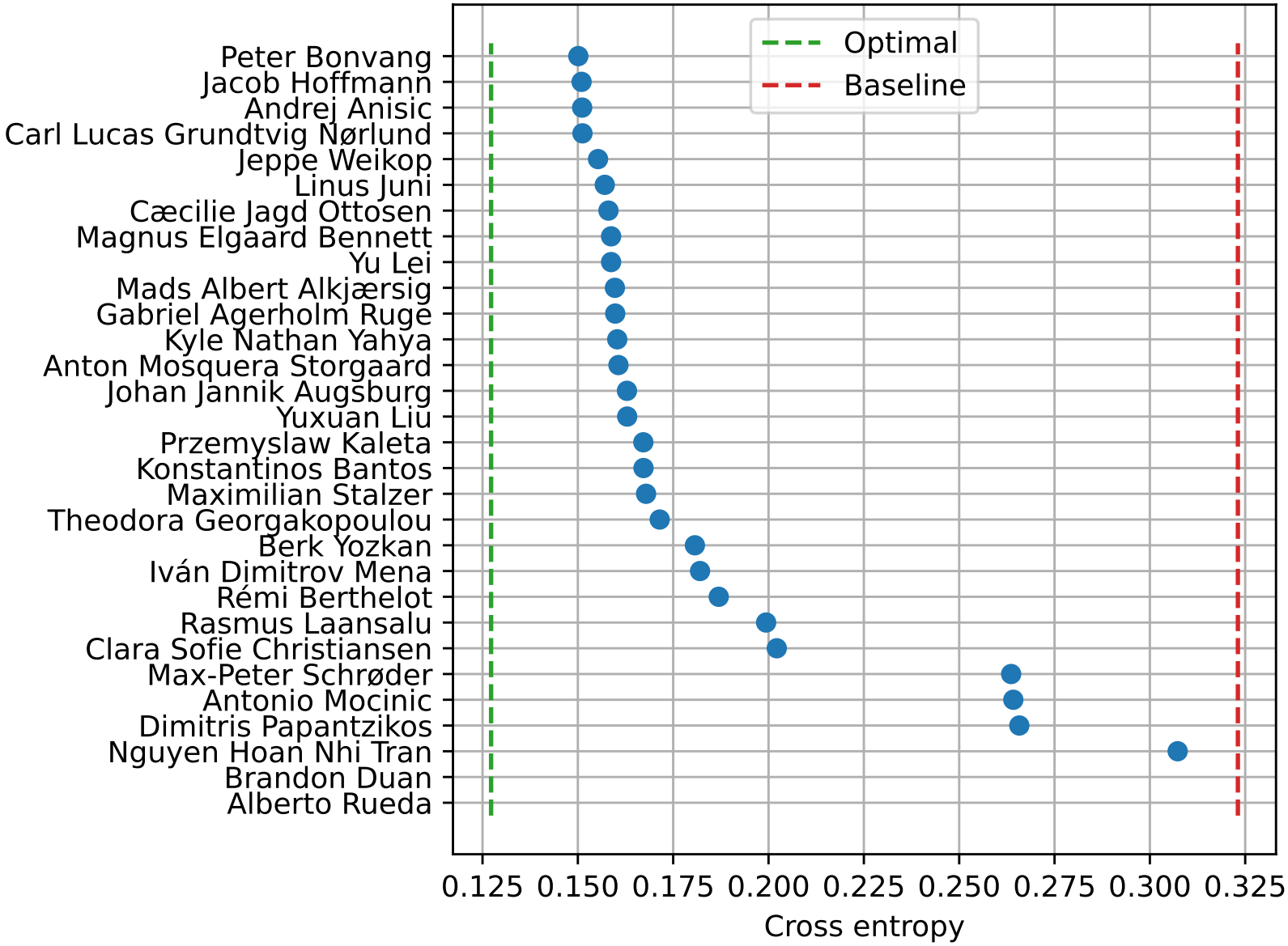
Module overview

Module 3: Graph models

1. *Graphs & node embeddings*
   Reading: Hamilton ch. 1–4
   Exercise 1
2. *Graph neural networks*
   Reading: Hamilton ch. 5–6
   Exercise 2
3. *Theory & generative models*
   Reading: Hamilton ch. 7–9
   Exercise 3
4. *Mini project 3*

Results from last week's competition

Last week's competition



Message passing on graphs

Why message passing

- There is no intrinsic notion of node order. The adjacecy matrices

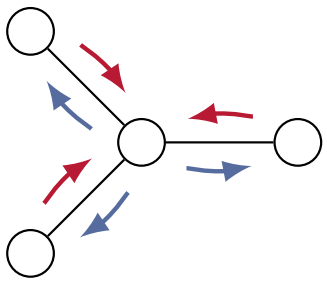
$$\boldsymbol{A} \quad \text{and} \quad \boldsymbol{A}^* = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^T$$

  describe the same graph for any permutation matrix $\boldsymbol{P}$.
- The way we represent and process a graph must not depend on any particular permutation.

$$f(\boldsymbol{A}) = f(\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^\top)$$

Message passing

1. *Initialization:* Each node has an initial state representation.
2. *Messages:* Each node sends a messages to its neighbors based on its current state.
3. *Aggregate:* Each node aggregates its received messages in combination with its previous state.
4. *Update:* Nodes update their state based on aggregated messages.
5. *Iteration:* Steps 2-4 are are repeated to let information propagate through the graph.
6. *Readout:* The final node states are used to make node level predictions, or aggregated to make graph level predictions.



Message passing

- *Initialization:* Node embeddings are initialized based on an embedding of node features (if available).

$$\boldsymbol{h}_u^{(0)} = \textsc{embed}(\boldsymbol{x}_u)$$

- *Message passing:* Node embeddings are updated based on aggregated information from neighbors and previous embedding.

$$\begin{aligned}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} &= \textsc{aggregate}^{(k)}(\{\boldsymbol{h}_v^{(k)} : v \in \mathcal{N}(u)\}) \\ \boldsymbol{h}_u^{(k+1)} &= \textsc{update}^{(k)}\left(\boldsymbol{h}_u^{(k)}, \boldsymbol{m}_{\mathcal{N}(u)}^{(k)}\right)\end{aligned}$$

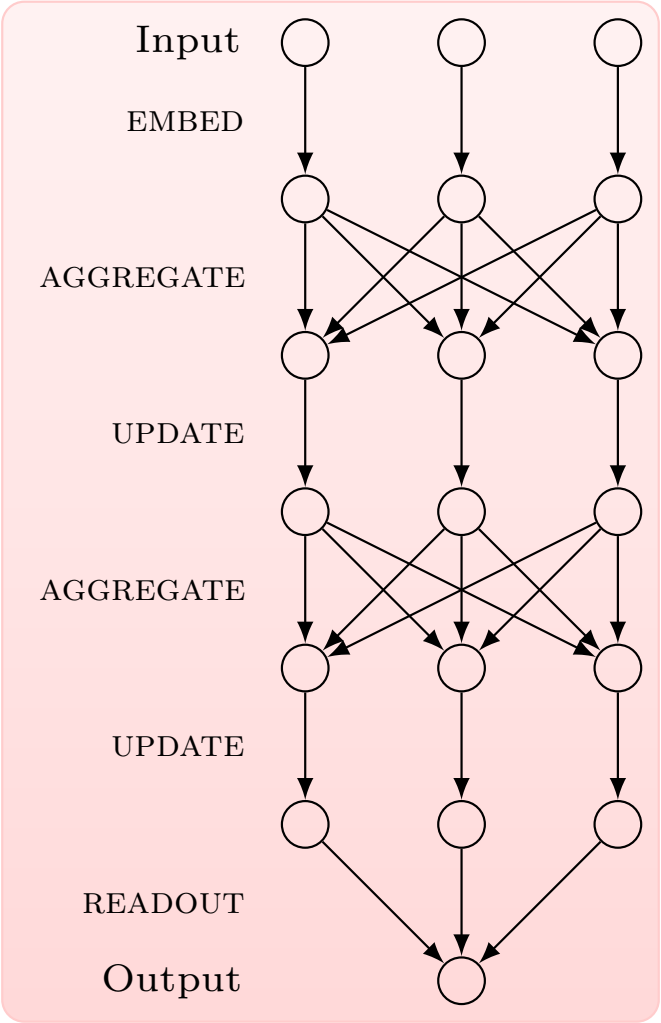
  Weight sharing or separate functions per layer.

- *Graph level readout:* If a graph-level prediction is required, node features are aggregated in a readout.

$$\boldsymbol{y} = \textsc{readout}\left(\{\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{|\mathcal{V}|}\}\right)$$

Message passing neural network

Example of a 2-layer GNN with graph-level readout on a graph with 3 nodes.



The basic GNN

An example of a very basic graph neural network:

$$\boldsymbol{h}_u^{(k)} = \overbrace{\sigma\left(\boldsymbol{W}_{\text{self}}^{(k)}\boldsymbol{h}_u^{(k-1)} + \boldsymbol{W}_{\text{neigh.}}^{(k)} \underbrace{\sum_{v\in\mathcal{N}(u)} \boldsymbol{h}_v^{(k-1)}}_{\text{AGGREGATE}} + \boldsymbol{b}^{(k)}\right)}^{\text{UPDATE}}$$

Message passing on graphs: Neighborhood aggregation

Aggregation

- Aggregation function should be permutation invariant.

$$f(x_1, x_2, \ldots, x_N) = f(x_{\pi_1}, x_{\pi_2}, \ldots, x_{\pi_N}) \quad \text{for any permutation } \pi$$

  Possibilities include sum, product, mean, max, etc.

- Normalization by node degree

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\boldsymbol{h}_v}{|\mathcal{N}(u)|}$$

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \frac{\boldsymbol{h}_v}{\sqrt{|\mathcal{N}(u)\mathcal{N}(v)|}}$$

  - Works better in some applications (e.g. high diversity in node degree)
  - Can lead to loss of information.

Aggregation

- *Set pooling*

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta \left(\sum_{v \in \mathcal{N}(u)} \mathrm{MLP}_\phi(\boldsymbol{h}_v) \right)$$

Universal function approximator.

Aggregation

- *Set pooling*

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta \left(\sum_{v \in \mathcal{N}(u)} \mathrm{MLP}_\phi(\boldsymbol{h}_v) \right)$$

Universal function approximator.

- *Janossy pooling* (randomized order)

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta \left(\frac{1}{|\Pi|} \underbrace{\sum_{\pi \in \Pi}}_{\text{Sum over a set of permutations}} \rho_\phi(\overbrace{[\boldsymbol{h}_{\pi_1}, \boldsymbol{h}_{\pi_2}, \ldots, \boldsymbol{h}_{\pi_{|\mathcal{N}(u)|}}]}^{\text{Concatenation of features}}) \right)$$

Average over many permutations of a permutation-sensitive function.

Aggregation

- *Set pooling*

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta\left(\sum_{v\in\mathcal{N}(u)} \mathrm{MLP}_\phi(\boldsymbol{h}_v)\right)$$

  Universal function approximator.

- *Janossy pooling* (randomized order)

$$\boldsymbol{m}_{\mathcal{N}(u)} = \mathrm{MLP}_\theta\left(\frac{1}{|\Pi|}\underbrace{\sum_{\pi\in\Pi}}_{\text{Sum over a set of permutations}}\rho_\phi(\overbrace{[\boldsymbol{h}_{\pi_1}, \boldsymbol{h}_{\pi_2}, \ldots, \boldsymbol{h}_{\pi_{|\mathcal{N}(u)|}}]}^{\text{Concatenation of features}})\right)$$

  Average over many permutations of a permutation-sensitive function.

- *Attention*

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v\in\mathcal{N}(u)} \alpha_{u,v}\boldsymbol{h}_v$$

$$\alpha_{u,v} = \underset{v\in\mathcal{N}(u)}{\mathrm{Softmax}}\left(\boldsymbol{h}_u^\top \boldsymbol{W}\boldsymbol{h}_v\right)$$

  Example here is bilinear attention—many other possibilities.

Transformer-style attention

- Key, query and value

$$\boldsymbol{k}_u = \boldsymbol{W}_{\text{key}} \boldsymbol{h}_u \qquad \boldsymbol{q}_u = \boldsymbol{W}_{\text{query}} \boldsymbol{h}_u \qquad \boldsymbol{v}_u = \boldsymbol{W}_{\text{value}} \boldsymbol{h}_u$$

- Scaled dot-product attention

$$\boldsymbol{m}_{\mathcal{N}(u)} = \sum_{v \in \mathcal{N}(u)} \alpha_{u,v} \boldsymbol{v}_v$$

$$\alpha_{u,v} = \underset{v \in \mathcal{N}(u)}{\text{softmax}} \left(\frac{\boldsymbol{k}_u^\top \boldsymbol{q}_v}{\sqrt{\underbrace{d}_{\text{Dimension of key and query vectors}}}} \right)$$

- Multi-head attention: Concatenate multiple of these mechanisms.

Message passing on graphs: Update methods

Self-loops

- Self-loops are a simple way to include the node itself in the neighborhood aggregation

$$\mathcal{N}(u) \cup \{u\}$$

  I.e., we consider the node u to be neighbor to itself.

- This way we can *skip the update*.

Over-smoothing

- Node representation can become very similar with increasing depth.
- *Intuitive reason:* Information from neighbors dominate.
- Methods for reducing over-smoothing[1]
  - Normalizing node features
  - Regularizing the training procedure
  - More advanced updates, that retain node information.

[1] See e.g. Rusch et al. A survey on oversmoothing in graph neural networks.

Concatenate, skip-connections, and gating

- *Concatenation*

$$\text{UPDATE}_{\text{concat.}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = [\text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}),\ \boldsymbol{h}_u]$$

Concatenate, skip-connections, and gating

- *Concatenation*

$$\text{UPDATE}_{\text{concat.}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = [\text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}),\ \boldsymbol{h}_u]$$

- *Skip-connections* (residual updates)

$$\text{UPDATE}_{\text{skip}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = \boldsymbol{h}_u + \text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)})$$

Concatenate, skip-connections, and gating

- *Concatenation*

$$\text{UPDATE}_{\text{concat.}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = [\text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}),\ \boldsymbol{h}_u]$$

- *Skip-connections* (residual updates)

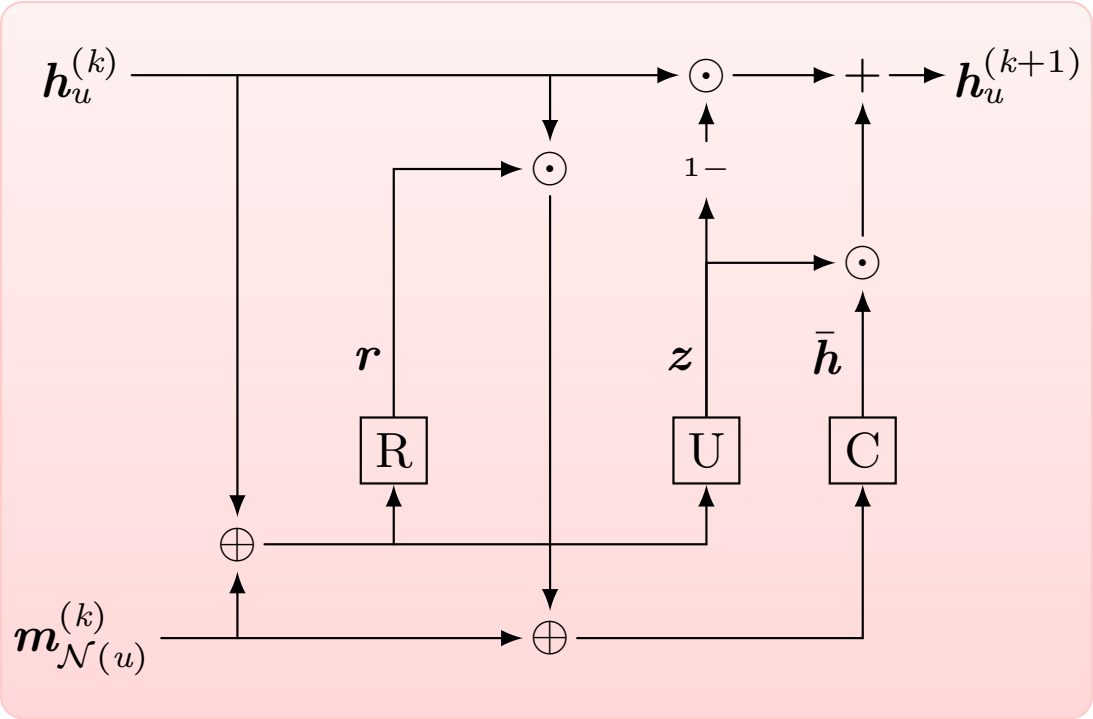
$$\text{UPDATE}_{\text{skip}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = \boldsymbol{h}_u + \text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)})$$

- *Gating* (linear interpolation)

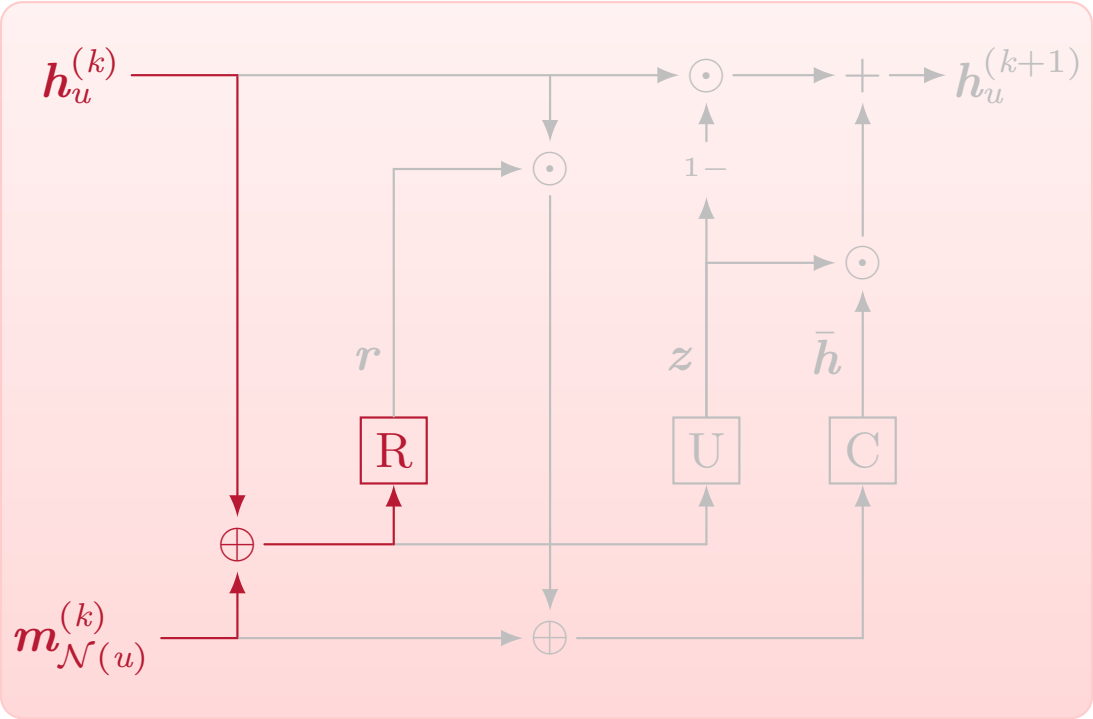
$$\text{UPDATE}_{\text{interp.}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)}) = \boldsymbol{\alpha} \odot \boldsymbol{h}_u + (1 - \boldsymbol{\alpha}) \odot \text{UPDATE}_{\text{base}}(\boldsymbol{h}_u, \boldsymbol{m}_{\mathcal{N}(u)})$$

Here $\boldsymbol{\alpha}$ is a gating vector which can be learned in many ways.

Gated recurrent unit (GRU)

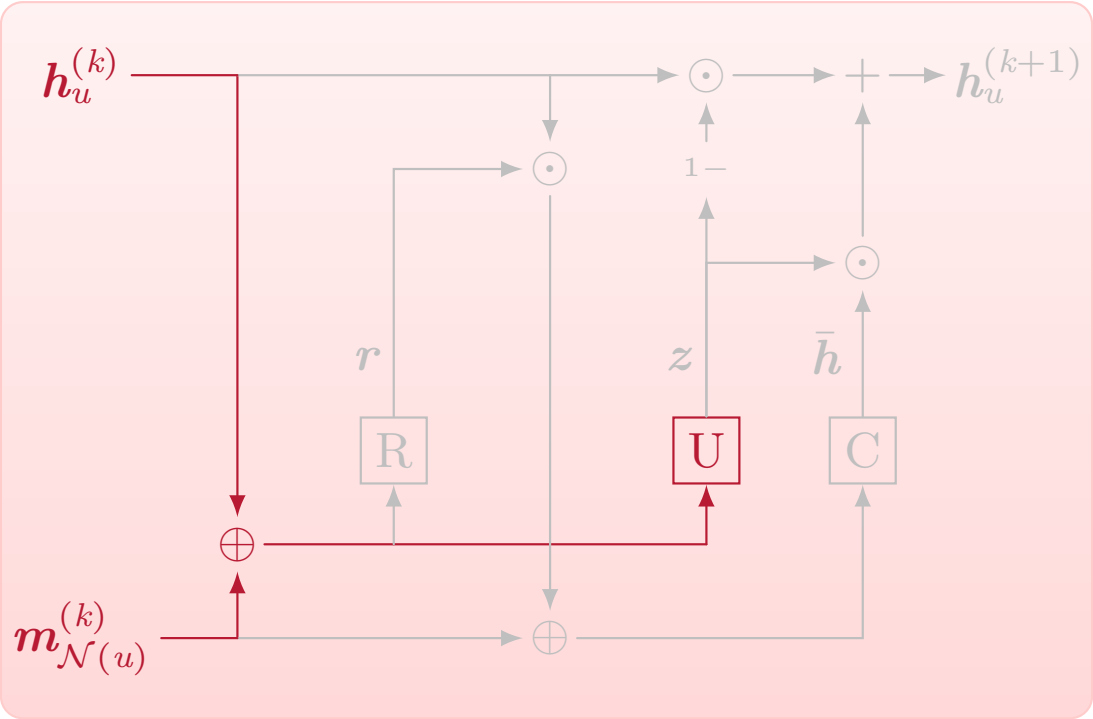


Gated recurrent unit (GRU)



Reset: $$\boldsymbol{r} = \sigma(\boldsymbol{W}_{mr}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hr}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_r)$$

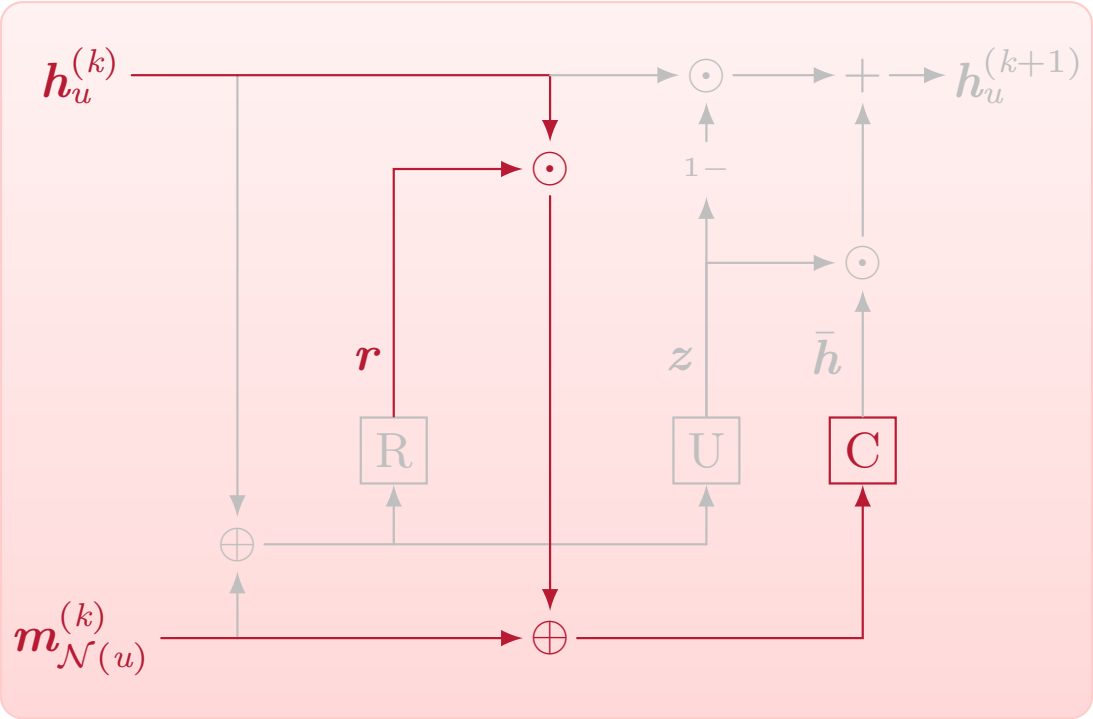
Gated recurrent unit (GRU)



Reset: $$\boldsymbol{r} = \sigma(\boldsymbol{W}_{mr}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hr}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_r)$$

Update: $$\boldsymbol{z} = \sigma(\boldsymbol{W}_{mz}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hz}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_z)$$

Gated recurrent unit (GRU)

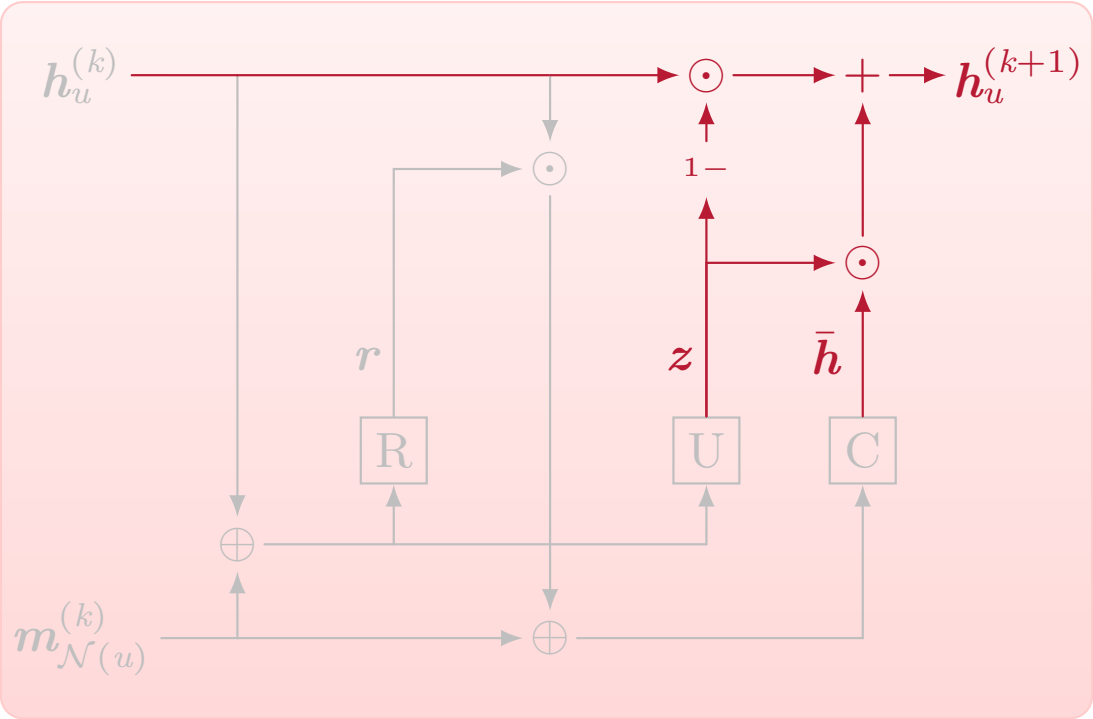


Reset: $$\boldsymbol{r} = \sigma(\boldsymbol{W}_{mr}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hr}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_r)$$

Update: $$\boldsymbol{z} = \sigma(\boldsymbol{W}_{mz}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hz}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_z)$$

Candidate: $$\bar{\boldsymbol{h}} = \tanh(\boldsymbol{W}_{mh}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hh}(\boldsymbol{r} \odot \boldsymbol{h}_u^{(k)}) + \boldsymbol{b}_h)$$

Gated recurrent unit (GRU)



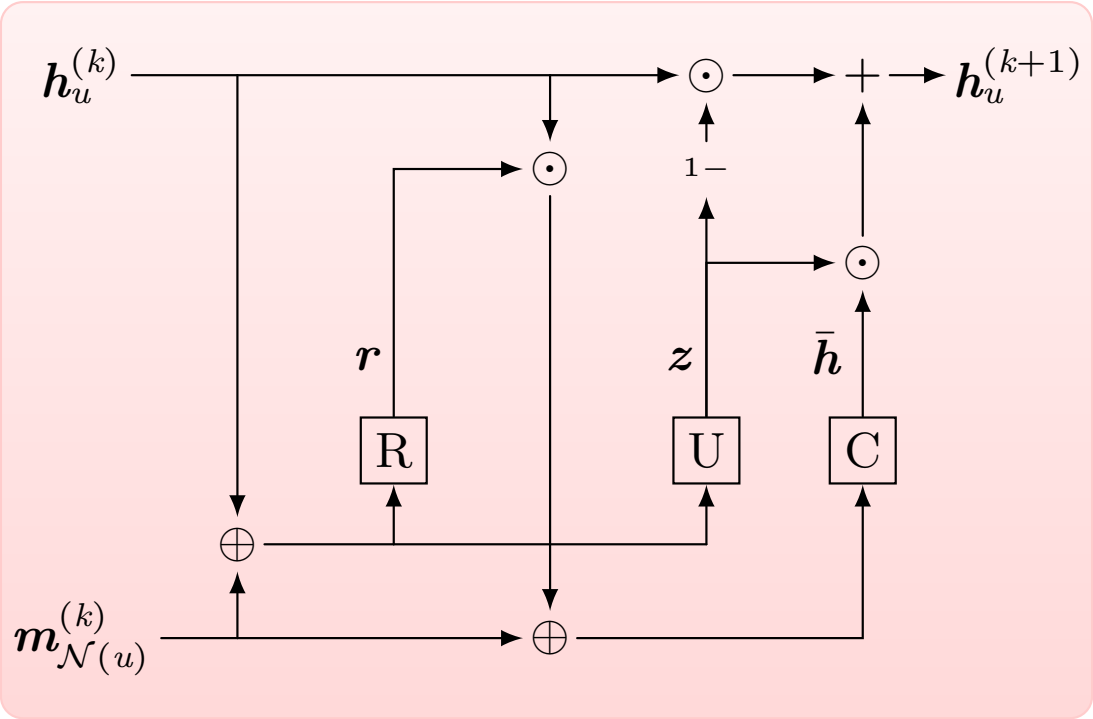
Reset: $$\boldsymbol{r} = \sigma(\boldsymbol{W}_{mr}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hr}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_r)$$

Update: $$\boldsymbol{z} = \sigma(\boldsymbol{W}_{mz}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hz}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_z)$$

Candidate: $$\bar{\boldsymbol{h}} = \tanh(\boldsymbol{W}_{mh}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hh}(\boldsymbol{r} \odot \boldsymbol{h}_u^{(k)}) + \boldsymbol{b}_h)$$

State: $$\boldsymbol{h}_u^{(k+1)} = \boldsymbol{z} \odot \bar{\boldsymbol{h}} + (1 - \boldsymbol{z}) \odot \boldsymbol{h}_u^{(k)}$$

Gated recurrent unit (GRU)



Reset: $\boldsymbol{r} = \sigma(\boldsymbol{W}_{mr}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hr}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_r)$

Update: $\boldsymbol{z} = \sigma(\boldsymbol{W}_{mz}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hz}\boldsymbol{h}_u^{(k)} + \boldsymbol{b}_z)$

Candidate: $\bar{\boldsymbol{h}} = \tanh(\boldsymbol{W}_{mh}\boldsymbol{m}_{\mathcal{N}(u)}^{(k)} + \boldsymbol{W}_{hh}(\boldsymbol{r} \odot \boldsymbol{h}_u^{(k)}) + \boldsymbol{b}_h)$

State: $\boldsymbol{h}_u^{(k+1)} = \boldsymbol{z} \odot \bar{\boldsymbol{h}} + (1 - \boldsymbol{z}) \odot \boldsymbol{h}_u^{(k)}$

Message passing on graphs: Graph pooling

Graph-level predictions

- Neural message passing produces node embedding
- If we want to make a graph-level prediction:
  - Set pooling from final node embedding to compute a graph-level embedding (similar to AGGREGATE).
  - Pooling from all layers, e.g. using a LSTM- or GRU-based recurrent neural network.

Generalized message passing

- *Node states*

$$\boldsymbol{h}_u^{(k+1)} = \text{UPDATE}_{\text{node}}\Big(\boldsymbol{h}_u^{(k)}, \underbrace{\text{AGG}_{\text{en}}(\{\boldsymbol{h}_{(u,v)}^{(k)} \forall v \in \mathcal{N}(u)\})}_{\text{Aggregate edges to node}}, \boldsymbol{h}_{\mathcal{G}}^{(k)}\Big)$$

- *Edge states*

$$\boldsymbol{h}_{(u,v)}^{(k+1)} = \text{UPDATE}_{\text{edge}}\Big(\boldsymbol{h}_{(u,v)}^{(k)}, \underbrace{\text{AGG}_{\text{ne}}(\{\boldsymbol{h}_u^{(k)}, \boldsymbol{h}_v^{(k)}\})}_{\text{Aggregate nodes to edge}}, \boldsymbol{h}_{\mathcal{G}}^{(k)}\Big)$$

- *Graph states*

$$\boldsymbol{h}_{\mathcal{G}}^{(k+1)} = \text{UPDATE}_{\text{graph}}\Big(\boldsymbol{h}_{\mathcal{G}}^{(k)}, \underbrace{\text{AGG}_{\text{eg}}(\{\boldsymbol{h}_u^{(k)} \forall u \in \mathcal{V}\})}_{\text{Aggregate nodes to graph}}, \underbrace{\text{AGG}_{\text{ng}}(\{\boldsymbol{h}_{(u,v)}^{(k)} \forall (u,v) \in \mathcal{E}\})}_{\text{Aggregate edges to graph}}\Big)$$

Message passing on graphs: Machine learning tasks on graphs

Machine learning tasks

Unsupervised

Community detection	Identify groups of connected nodes.
Link prediction	Predict formation of new edges.
Node embedding	Learn low-dimensional representations of nodes.

Supervised

Node/edge classification	Predict node/edge labels
Graph property prediction	Predict properties of the entire graph.

Generative

Graph generation	Generate new graphs that resemble real-world data.
Graph completion	Predict missing nodes/edges in incomplete graph.

Graph neural networks can be used in all these tasks.

Transduction and induction

In a *node prediction task* we can distinguish 3 types of nodes

1. *Training nodes:* Used in message passing and to compute loss.
   These are purely training data.
2. *Transductive nodes:* Nodes to train message passing but not in loss.
   We can make node level predictions, having learned their specific embedding.
3. *Inductive nodes:* Not used in training.
   We can make node level predictions by using the trained GNN to infer their embedding.
   Contrast to shallow embeddings, where induction is not possible.

Geometric embedding

Handling symmetries

Consider graphs embedded in a vector space (nodes have coordinates.)

- Physical features change deterministicaly under certain transformations.
  We do not need to learn this.
  - E.g. energy of a molecule does not change with translation and rotation.
  - Forces on atoms translate and rotate with the molecule.
- Coordinate systems are arbitraty.
  The model should not be sensitive to choice of coordinate system.

Symmetry-aware modeling

Three general ways to handle symmetries

1. Data augmentation (brute-force)
2. Symmetry-aware descriptors (pre-processing, constraining the data space)
3. Symmetry-aware models (built-in, constraining the function space)
   We focus on this approach

Invariance and equivariance

Example: Invariance and equivariance in sequence translation:

- *Invariant:* Output does not depend on input translation.

$$[1, 2, 3, 0, 0] \to [0, 1, 0, 0, 0]$$
$$[0, 1, 2, 3, 0] \to [0, 1, 0, 0, 0]$$
$$[0, 0, 1, 2, 3] \to [0, 1, 0, 0, 0]$$

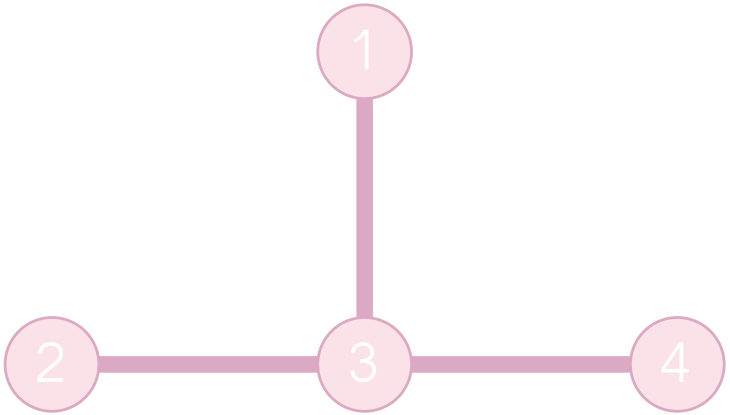
- *Equivariant:* Output translates with input translation.

$$[1, 2, 3, 0, 0] \to [0, 1, 0, 0, 0]$$
$$[0, 1, 2, 3, 0] \to [0, 0, 1, 0, 0]$$
$$[0, 0, 1, 2, 3] \to [0, 0, 0, 1, 0]$$

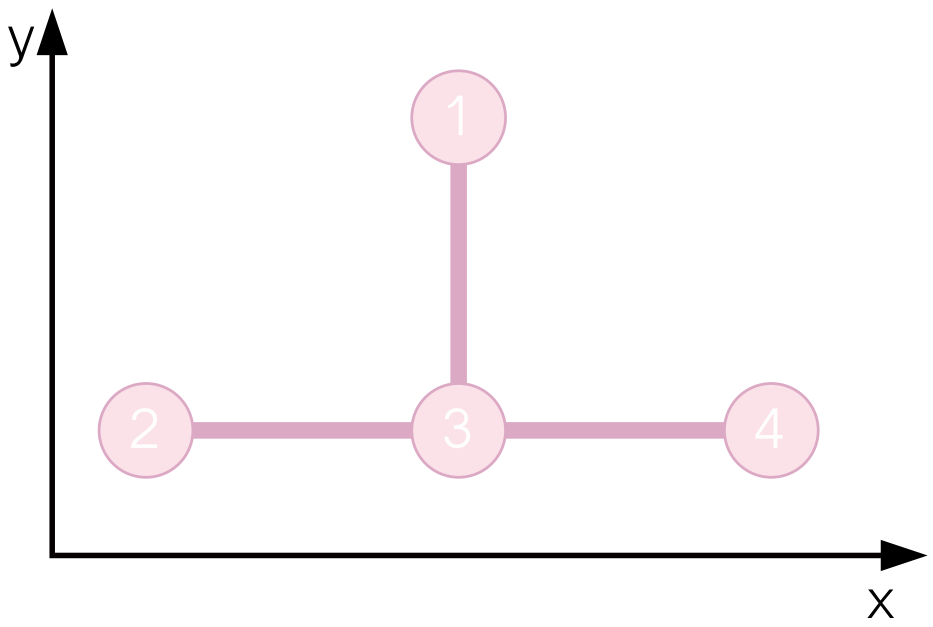
Example: Molecular graph classification

- Data is a set of graphs (molecules)
- Each node has a position in a vector space
- *We want the GNN to be invariant to rotation and translation*

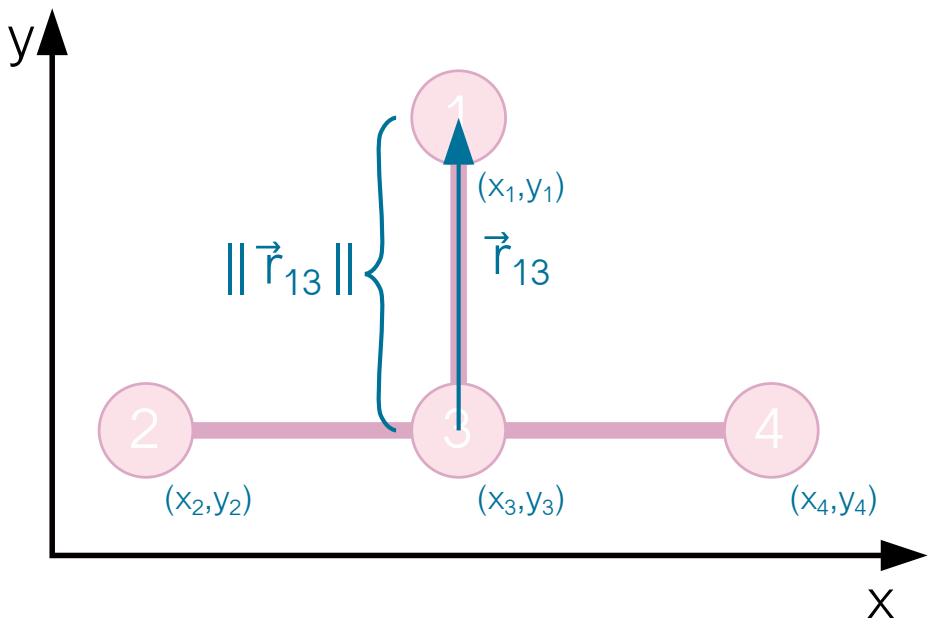
Graph embedded in vector space



Graph embedded in vector space



Graph embedded in vector space



Rotation and translation of graph embedded in vector space

- Invariant

$$\vec{f}\big(T(\vec{h})\big) = \vec{f}(\vec{h})$$

- Equivariant

$$\vec{f}\big(T(\vec{h})\big) = T'\big(\vec{f}(\vec{h})\big)$$

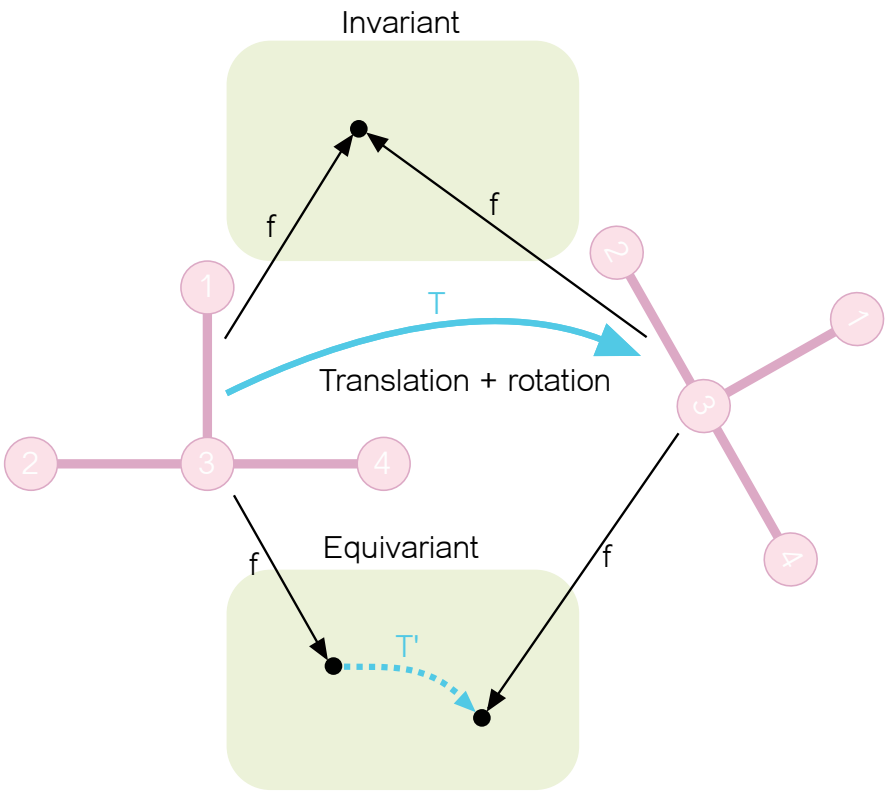
$\vec{h}$ A node- or graph-level feature

f Operation performed on the feature

T Rotation+translation operator

T' Corresponding output operator

$T = T'$ when x and $f(x)$ are in the same vector space.



Adapted from Elise van der Pol, Daniel E. Worrall

How to make a invariant/equivariant GNN

- Initialize node states with *scalar* and *vector* features.
- Let node state consist of scalar and vectorial parts.
- Construct AGGREGATE, UPDATE, and READOUT functions using only invariant/equivariant operations.

This ensures that the entire GNN is invariant/equivariant.

Vector features

Examples of invariant/equivariant operations

$a \cdot \vec{v}$	Scale
$\vec{v}_1 + \vec{v}_2$	Addition/subtraction
$\|\|\vec{v}\|\|$	Norm
$\vec{v}_1 \cdot \vec{v}_2$	Dot product $= \|\vec{v}_1\| \cdot \|\vec{v}_2\| \cdot \cos(\theta)$
$\vec{v}_1 \times \vec{v}_2$	Cross product (3-d)
$\|\vec{v}_1 \times \vec{v}_2\|$	Cross product (2-d) $= \|\vec{v}_1\| \cdot \|\vec{v}_2\| \cdot \sin(\theta)$

Which operations are rotationally invariant and which are equivariant?

Why vector features?

- It is limited, what can be expressed with scalar features
- Vector features encode local geometry in more detail
- Even if we want an invariant GNN, vector features are useful.

Pytorch implementation

Python tools for implementing GNN

- *PyG (PyTorch Geometric)* is a library built upon PyTorch to easily write and train GNNs
- However, basic GNNs are not so hard to implement in plain PyTorch.

Here, we opt for the latter approach for pedagogical reasons.

TORCH.TENSOR.INDEX_ADD_

Tensor.index_add_(*dim*, *index*, *source*, *, *alpha=1*) → Tensor

Accumulate the elements of `alpha` times `source` into the `self` tensor by adding to the indices in the order given in `index`. For example, if `dim == 0`, `index[i] == j`, and `alpha=-1`, then the `i` th row of `source` is subtracted from the `j` th row of `self`.

The `dim` th dimension of `source` must have the same size as the length of `index` (which must be a vector), and all other dimensions must match `self`, or an error will be raised.

For a 3-D tensor the output is given as:

```
self[index[i], :, :] += alpha * src[i, :, :]  # if dim == 0
self[:, index[i], :] += alpha * src[:, i, :]  # if dim == 1
self[:, :, index[i]] += alpha * src[:, :, i]  # if dim == 2
```

NOTE

This operation may behave nondeterministically when given tensors on a CUDA device. See Reproducibility for more information.

Parameters

- **dim** (*int*) – dimension along which to index
- **index** (*Tensor*) – indices of `source` to select from, should have dtype either *torch.int64* or *torch.int32*
- **source** (*Tensor*) – the tensor containing values to add

Keyword Arguments

alpha (*Number*) – the scalar multiplier for `source`

Exercises

Exercises

A Invariant aggregation functions

B Simple graph neural networks

C Programming exercise (GNN for graph-level classification)